Experiment 06: Docker Images and Deploy Containerized Applications using Docker

Experiment No.	06
Mapped CO	CO4
Aim	To create Docker images and deploy containerized applications using Docker.
Tools / Software	Docker Engine 24 or later; Ubuntu 22.04+ / Windows 11 with WSL2; Docker Hub account; A sample web application

Theory

In DevOps, application deployment should be repeatable, portable and easy to verify. Docker supports this requirement by packaging an application, runtime, dependencies and configuration into a deployable Docker image. The same image can be executed as a container on any compatible host.

A Docker image is a versioned, read-only package created from a Dockerfile. A container is the running instance of that image. For deployment, images are usually tagged with version names, pushed to a registry such as Docker Hub, pulled on a target machine and executed using docker run with suitable port mapping and restart options.

Common Dockerfile instructions such as FROM, WORKDIR, COPY, RUN, EXPOSE and CMD define how the image is built and how the application starts. During deployment, options such as -d, --name, -p and --restart help run the application in background mode, expose it to the host and keep it available after service restarts.

Procedure

1. Install Docker Engine and verify the installation using docker --version and docker info.
2. Create a project directory named docker-deployment-demo and place the sample application files inside it.
3. Create app.py with a simple Flask web route and create requirements.txt with the required dependency.
4. Write a Dockerfile defining the base image, working directory, dependency installation, exposed port and startup command.
5. Build a versioned Docker image using docker build and tag it as mywebapp:1.0.
6. Run the image locally as a container with port mapping and verify the output in a browser or using curl.
7. Tag the image for Docker Hub or registry-based deployment and optionally push it to the registry.
8. Deploy the image as a detached container using docker run with --name, -p and --restart options.
9. Verify deployment using docker ps, docker logs and browser/curl output.
10. Modify the application, build a new version such as mywebapp:2.0 and redeploy by replacing the old container.
11. Stop and remove containers/images after completion to clean up resources.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello from Docker deployed application!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, deploy, verify and redeploy commands

```
docker --version
docker build -t mywebapp:1.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:1.0
docker ps
curl http://localhost:8080
docker logs mywebapp
docker tag mywebapp:1.0 <dockerhub-user>/mywebapp:1.0
docker push <dockerhub-user>/mywebapp:1.0
docker stop mywebapp && docker rm mywebapp
docker build -t mywebapp:2.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:2.0
```

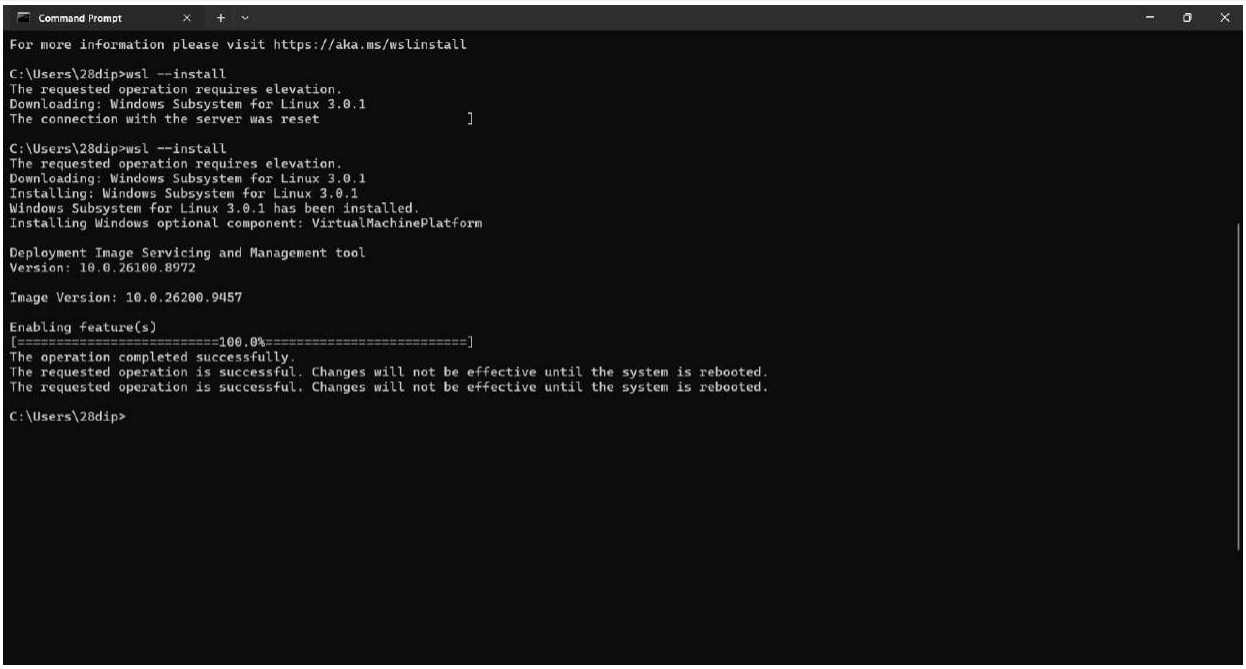
Observation / Experiment Output

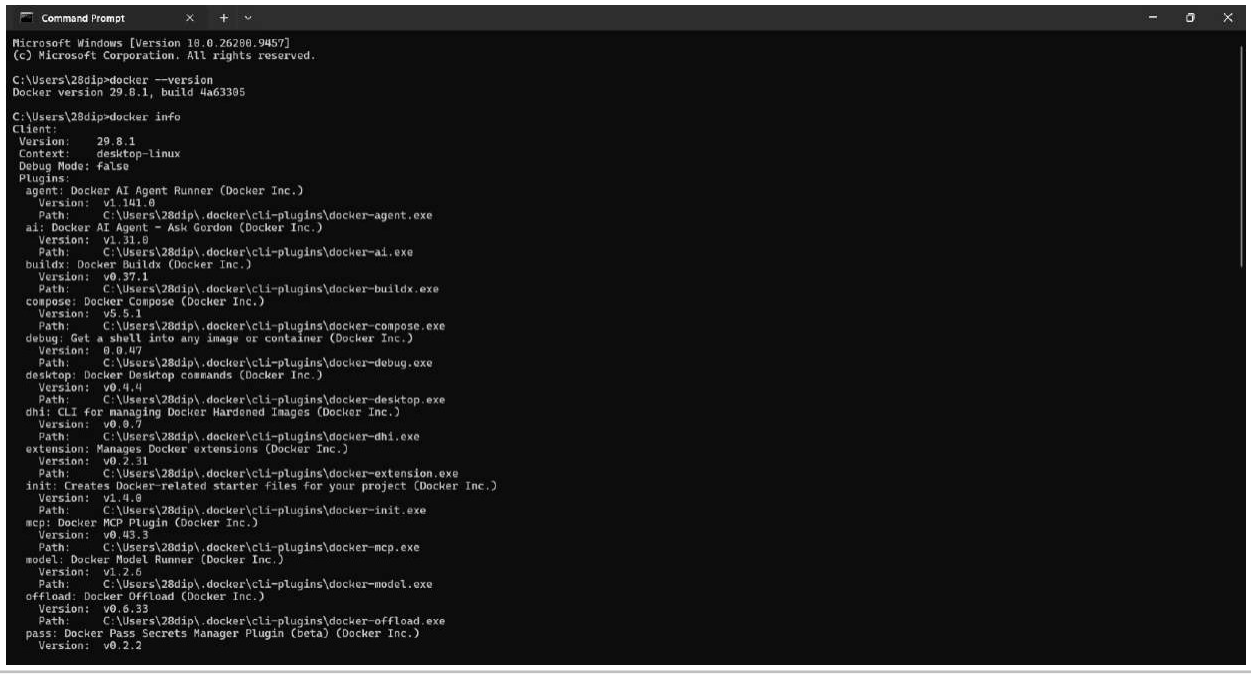
Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker image creation, container deployment and verification. Keep screenshots clear and readable.

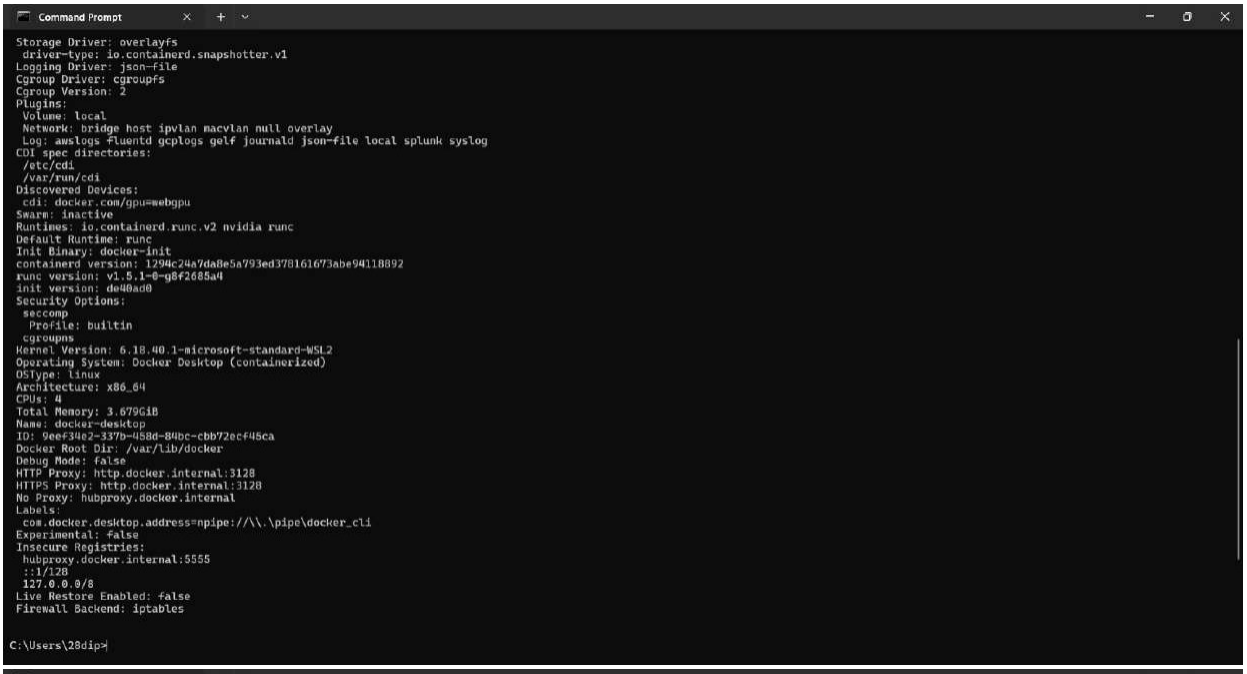
Name	Roll No.	Batch	Date
Vighnesh Pingale	24107108	B2	03-10-2026

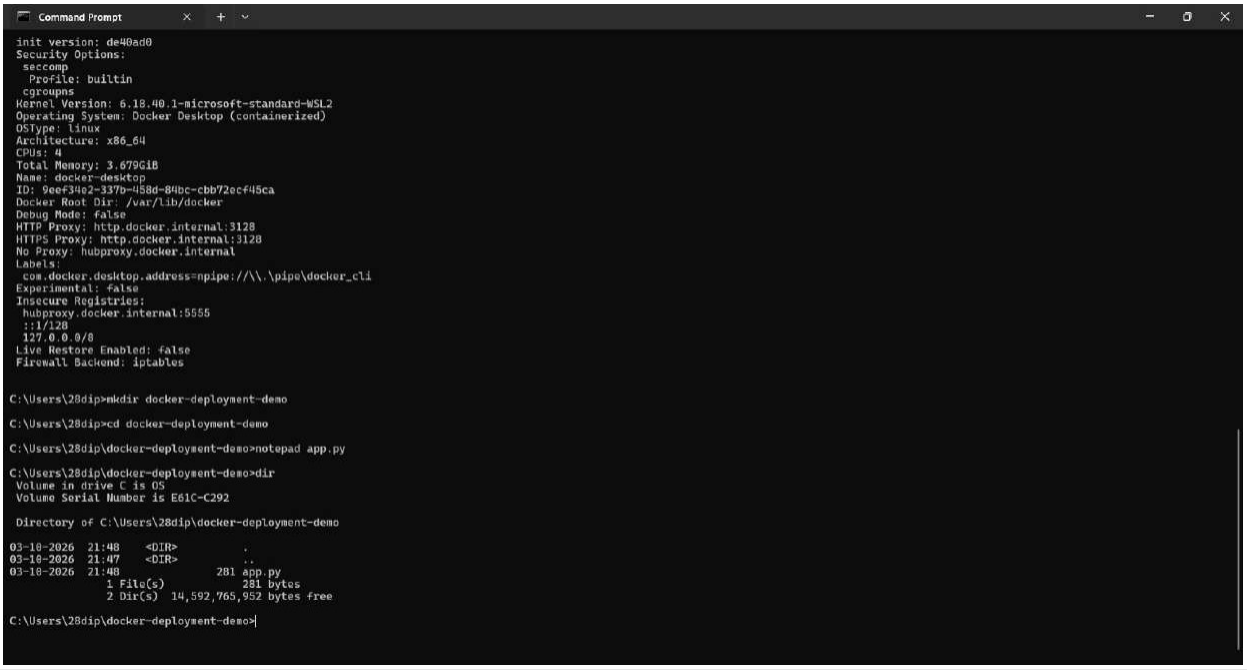
1. Docker Installation and Project Setup Output

Paste screenshot/output showing Docker version, Docker info and the created project directory.



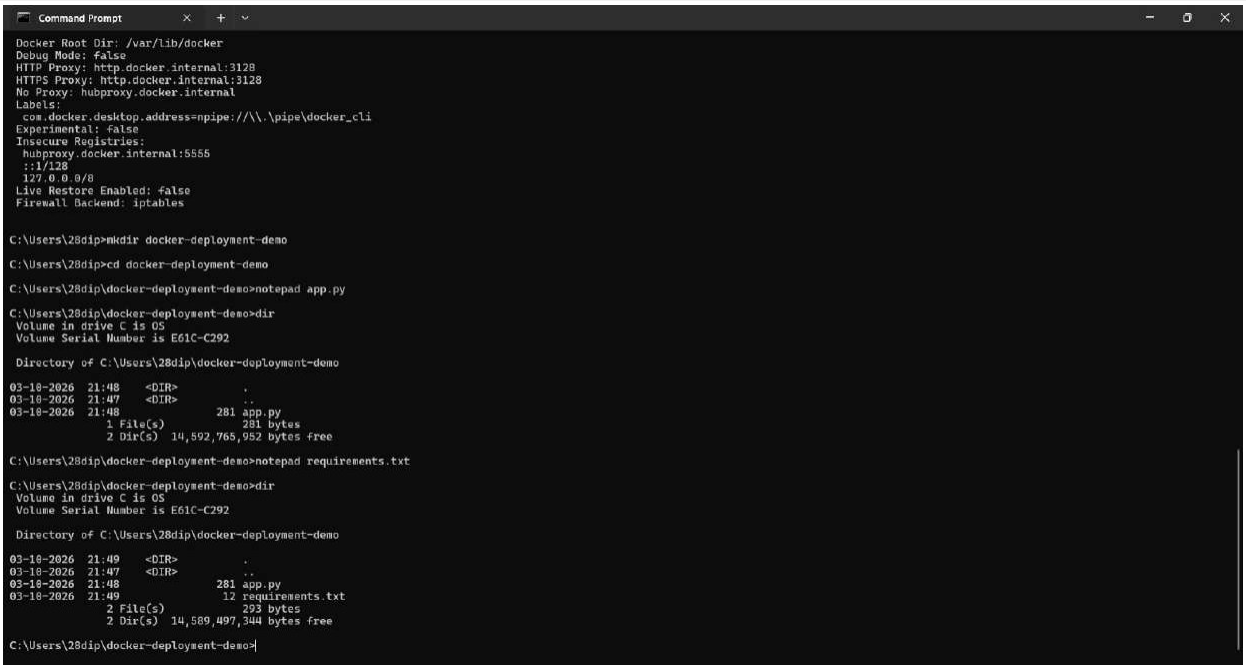


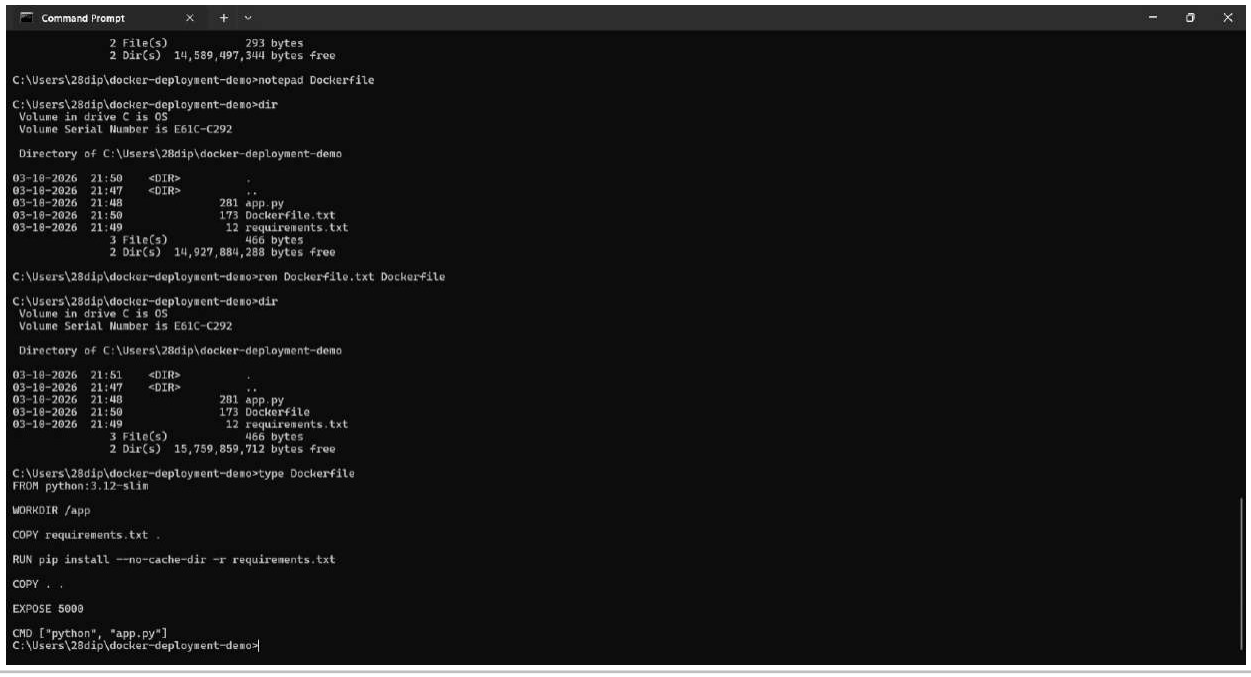

Command Prompt
Storage Driver: overlayfs
driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
Volume: local
Network: bridge host ipvlan macvlan null overlay
Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
/etc/cdi
/var/run/cdi
Discovered Devices:
cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: io.containerd.runc.v2 nvidia runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 1294c24a7da8e5a793ed37816167 3abe94118892
runc version: v1.5.1-0-g8f2685a4
init version: de40ad0
Security Options:
seccomp
Profile: builtin
cgroupns
Kernel Version: 6.18.40.1-microsoft-standard-WSL2
Operating System: Docker Desktop (containerized)
OSType: linux
Architecture: x86_64
CPUs: 4
Total Memory: 3.679GiB
Name: docker-desktop
ID: 9eef34e2-337b-458d-84bc-cbb72ecf45ca
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
hubproxy.docker.internal:5555
::1/128
127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
C:\Users\28dip>


Command Prompt
init version: de40ad0
Security Options:
seccomp
Profile: builtin
cgroupns
Kernel Version: 6.18.40.1-microsoft-standard-WSL2
Operating System: Docker Desktop (containerized)
OSType: linux
Architecture: x86_64
CPUs: 4
Total Memory: 3.679GiB
Name: docker-desktop
ID: 9eef34e2-337b-458d-84bc-cbb72ecf45ca
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
hubproxy.docker.internal:5555
::1/128
127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
C:\Users\28dip>mkdir docker-deployment-demo
C:\Users\28dip>cd docker-deployment-demo
C:\Users\28dip\docker-deployment-demo>notepad app.py
C:\Users\28dip\docker-deployment-demo>dir
Volume in drive C is OS
Volume Serial Number is E61C-C292
Directory of C:\Users\28dip\docker-deployment-demo
03-10-2026 21:48 <DIR> .
03-10-2026 21:47 <DIR> ..
03-10-2026 21:48 281 app.py
1 File(s) 281 bytes
2 Dir(s) 14,592,765,952 bytes free
C:\Users\28dip\docker-deployment-demo>

2. Application Source and Dockerfile Output

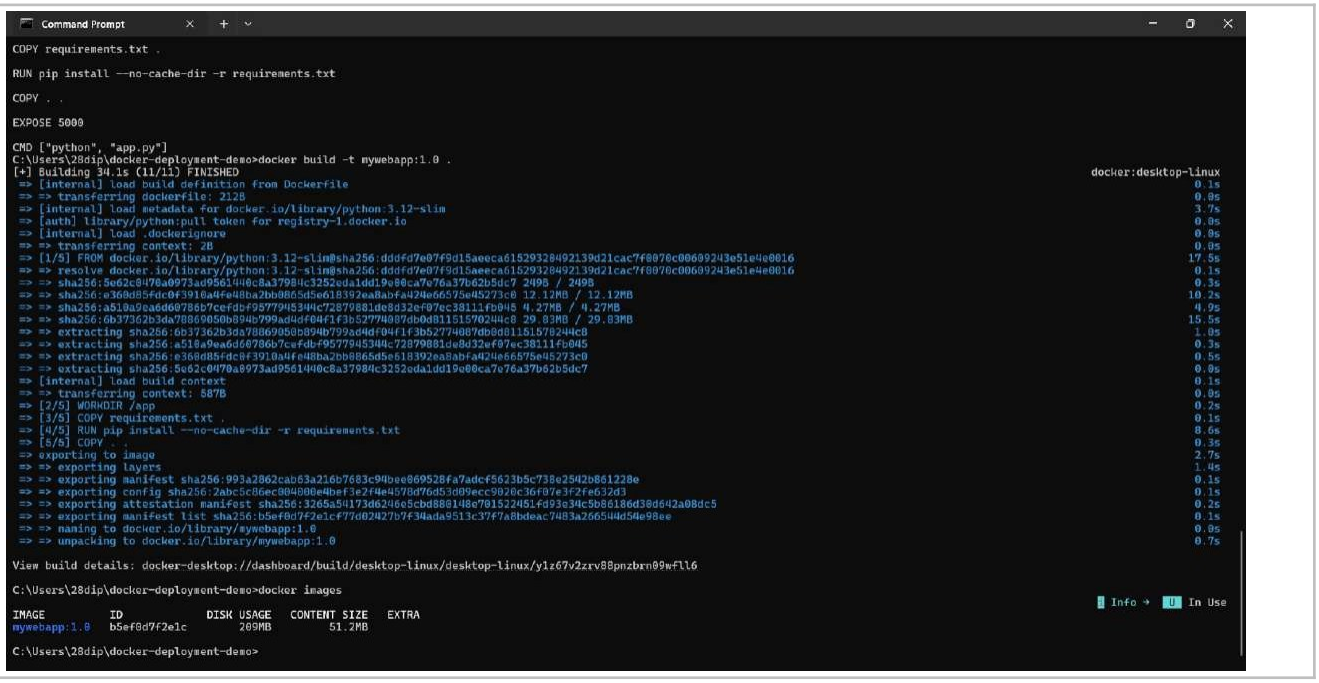
Paste screenshot/output showing app.py, requirements.txt and Dockerfile contents.





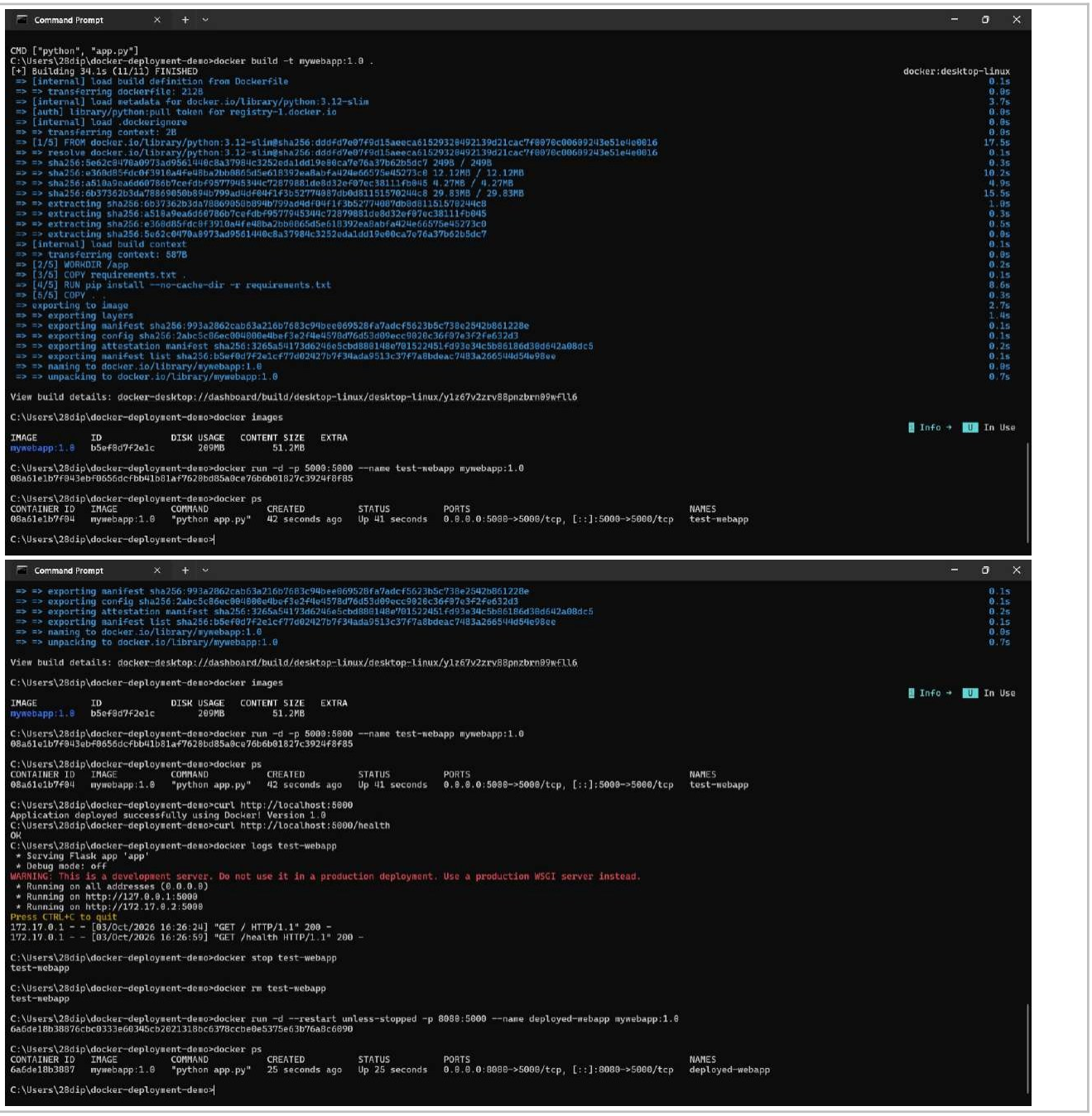
3. Docker Image Build and Tag Output

Paste screenshot/output showing successful docker build command and image tag mywebapp:1.0.



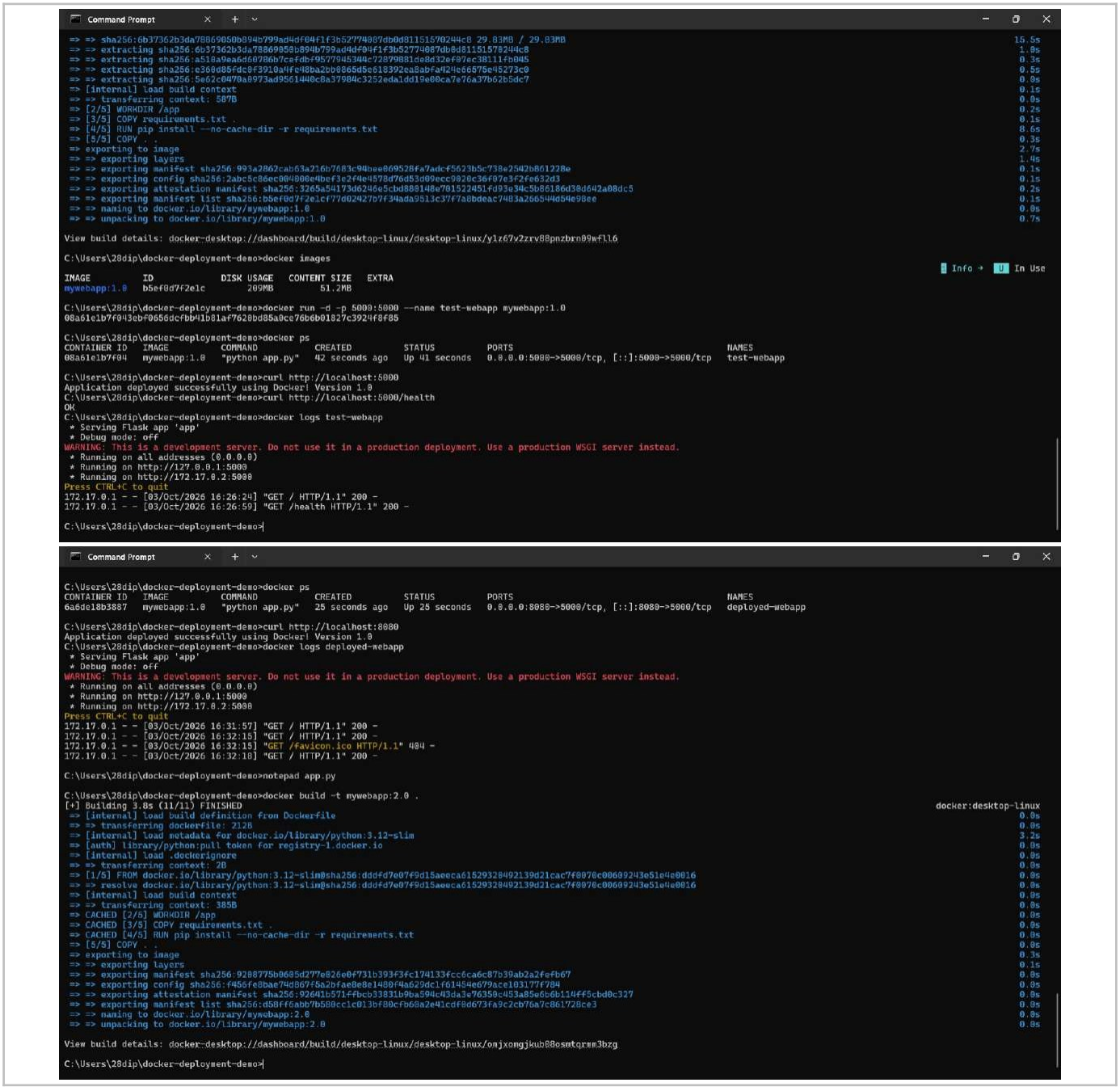
4. Container Deployment and Port Mapping Output

Paste screenshot/output showing docker run command, container name, restart policy and port mapping.



5. Application Access and Logs Output

Paste screenshot/output showing browser/curl response, docker ps and docker logs output.



6. Docker Hub Push / Pull or Redeployment Output

Paste screenshot/output showing image push/pull or updated version deployment such as mywebapp:2.0.



GitHub Repository / Lab Folder Link:

https://github.com/Vighnesh279/My-project (folder: EXP-06-Docker-Deployment)

Docker Hub Repository Link (Optional):

Not applicable - image was deployed locally (push to Docker Hub is optional for this experiment).

Final Observation Statement:

```
Docker was installed and verified, and the image mywebapp:1.0 was built from the Dockerfile
of a Flask application. It was deployed as a container with port mapping (8080:5000) and --
restart unless-stopped, verified using curl, docker ps and docker logs, and redeployed as
mywebapp:2.0.
```

Expected Output

Docker should be installed and working. The sample web application and Dockerfile should be created successfully. A versioned Docker image should be built, deployed as a container with port mapping, and verified through browser/curl output. Container status and logs should be visible using docker ps and docker logs. Optional registry push/pull or updated redeployment should also be demonstrated.

Conclusion

Docker images were created and used to deploy a containerized web application. The experiment demonstrated image building, tagging, container deployment, port mapping, log inspection and redeployment of an updated version.

Viva / Review Questions

1. What is the difference between a Docker image and a Docker container?
2. Why are version tags useful during Docker-based deployment?
3. Explain the role of port mapping in docker run.
4. What is the purpose of the --restart option during container deployment?

Marks: ____ / 10 **Date: _____** **Signature of Faculty: _____**